Report

Of

Software Engineering & Architecture Internship at YuvaIntern

Submitted

To

School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

B.Tech CSE

By

Roll Number of Student: 2410031061

Name of Student: VINEET TIWARI

Batch: 2024-2028

2026-27

CANDIDATE'S DECLARATION

I, VINEET TIWARI**,** do hereby declare that the internship report titled **"Software Engineering & Architecture"** has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in CSE. All the references have been quoted and I have not taken as such any material from any other source. I affirm to you that this report is my own and has not been submitted to any other institute for any degree/diploma requirement and further I shall be solely responsible for any kind of copyright violation in this regard.

Signature
Student Name : Vineet Tiwari
Roll No. : 2410031061

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their aspiring guidance, invaluably constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work.
I express my gratitude to my parents for their blessings.

Signature
Student Name : Vineet Tiwari
Roll No. : 2410031061

Dear Vineet Tiwari,

We are pleased to extend to you this formal offer for an Internship position as **Lead Software Engineer hyderabad, in** at **YuvaIntern**. We were impressed with your background, skills, and enthusiasm, and we believe you will be a valuable addition to our team.

Internship Details:
• **Start Date:** August 04, 2026
• **End Date:** September 01, 2026
• **Duration:** 4 weeks
• **Location:** Remote

Key Responsibilities:
As a **Lead Software Engineer hyderabad, in** intern, you will work on tasks and projects related to Internship program for Lead Software Engineer hyderabad, in..

Benefits & Learning Opportunities:
Throughout the duration of your internship, you will have the opportunity to:

- Develop practical skills in [relevant skills/tools].
- Gain hands on experience in [specific tasks/projects].

As part of joining our team, you may be required to sign a confidentiality or non-disclosure agreement. You are also expected to comply with all applicable company policies and regulations, which will be provided to you upon commencement.

Kounal Gupta
Founder, YuvaIntern

CERTIFICATE OF EXPERIENCE

To Whomsoever It May Concern

This is to certify that **Vineet Tiwari** successfully completed an internship with **YuvaIntern** in the role of **Lead Software Engineer hyderabad, in**.

Throughout the tenure of this internship, Vineet demonstrated consistent dedication, professionalism, and a strong willingness to learn and grow. Their contributions were valued by the entire team at YuvaIntern, and they conducted themselves with integrity and commitment throughout the program.

We commend Vineet's hard work and wish them continued success in all future endeavors.

Certificate No: YI/2026/184183/409162
Date of Issue: September 01, 2026

FOR AND ON BEHALF OF YUVAINTERN

Kounal Gupta
Founder, YuvaIntern.com

Henry Harvin America Head Office
Henry Harvin Inc
8 The Green, # 19614 Dover,
DE 19901, United States.

Henry Harvin Asia Pacific Office
Henry Harvin India Education LLP
Henry Harvin House,
B-12, Sector-6, Noida(UP), India- 201301

Henry Harvin Middle East Office
Henry Harvin Co. L.L.C.
2703, Blue Matrix, 27th floor The
Prime Tower, Business Bay, Dubai, UAE

Table of Contents

Project Description

1. Introduction

This report presents the work completed during a four-week internship undertaken as a Software Engineering Trainee under the AICTE–EduSkills Virtual Internship Program, in the domain of Software Engineering & Architecture. The internship was carried out as part of the B.Tech CSE programme at IILM University, Greater Noida, and required the design, planning, review, and performance evaluation of a proposed high-traffic e-commerce platform.

The primary objective of the internship was to gain practical exposure to how large-scale software systems are engineered in the industry, going beyond writing individual pieces of code to understanding how architecture, planning, code quality, and performance are connected to one another across the software development lifecycle. The project used for this purpose — a Scalable E-Commerce Platform based on Microservices Architecture — was chosen because e-commerce systems naturally involve many of the challenges associated with high-traffic applications, such as concurrent users, multiple interdependent services, and the need for strong security and reliability.

The internship was structured over four weeks, with each week building upon the previous one. Week 1 focused on designing a scalable microservices-based architecture for the platform. Week 2 translated that architecture into a practical implementation plan with task breakdowns, timelines, and risk management. Week 3 involved a representative code review and quality assurance exercise to identify and address common issues in the proposed services. Week 4 focused on designing a performance testing and optimization strategy to ensure the platform could handle high traffic reliably.

Together, these four stages reflect a realistic view of how software architecture decisions influence project planning, how code quality affects long-term maintainability, and how performance testing validates whether a system meets its non-functional requirements. This report documents the work carried out in each of these stages and the learning outcomes derived from them.

2. Organization Profile

YuvaIntern is a virtual internship and job-simulation platform designed to bridge the gap between theoretical education and real-world professional responsibilities. According to its official website, the platform follows a “learning by doing” approach, where internship simulations are aligned with actual job descriptions and focus on practical tasks and key responsibilities.

YuvaIntern offers opportunities across multiple fields, including technology, finance, marketing, HR, design, and other sectors. Its technology-related areas include software development, data analytics, and technical support. The platform allows students to complete internships virtually and at their own pace, without requiring relocation.

YuvaIntern states that it is an initiative of Henry Harvin Education, a global higher-education and EdTech organization. The platform also states that its simulations are developed with industry experts and that it works with NSDC and industry partners. The internship follows a structured process involving enrollment, an offer letter, completion of job-simulation tasks, evaluation/feedback, and certification. This model provides students with practical exposure while allowing them to understand the responsibilities associated with a professional role.

For this internship, YuvaIntern provided the platform through which I completed my Software Engineering Trainee role, focusing on software architecture, project planning, code review and quality assurance, and performance testing and optimization.

3. Problem Statement

Modern e-commerce platforms are expected to serve a large and continuously growing number of users, many of whom interact with the system at the same time. This creates several challenges that a well-designed system must address, including handling concurrent requests without degrading response times, scaling individual parts of the application independently as demand grows, and maintaining high availability even when parts of the system fail.

In addition to scalability, an e-commerce platform must remain reliable and secure, since it handles sensitive information such as user accounts and payment details. It must also remain maintainable over time, as new features are continuously added and existing ones are modified. Traditional monolithic architectures, where all functionality is built and deployed as a single unit, often struggle to meet these requirements. A single point of failure can bring down the entire application, scaling requires duplicating the whole system rather than just the parts under load, and the tight coupling between modules makes the codebase harder to understand, test, and maintain as it grows.

To address these challenges, a Microservices Architecture was considered for the proposed e-commerce platform. By dividing the system into independent, loosely coupled services — each responsible for a specific business capability such as user management, product catalog, cart, orders, inventory, payments, and notifications — it becomes possible to scale, deploy, and maintain each part of the system independently. This forms the basis of the architecture designed and documented during the internship.

4. Project Objectives

1. To design a scalable e-commerce architecture based on microservices principles.
2. To understand how microservices divide responsibilities and communicate with one another.
3. To develop a practical implementation plan covering tasks, timelines, and resource allocation.
4. To identify code quality, security, and maintainability issues through a representative code review.
5. To define suitable quality assurance practices and test cases for the proposed system.
6. To design a performance testing strategy covering load, stress, spike, and endurance scenarios.
7. To identify likely performance bottlenecks in a high-traffic e-commerce system.
8. To propose appropriate optimization methods based on testing outcomes.
9. To gain an overall understanding of the software engineering lifecycle.
10. To understand the relationship between architecture, code quality, and system performance.

5. Scope of the Project

The scope of the proposed system covers the core functional areas typically required by an e-commerce platform, along with the supporting technical concerns needed to operate them reliably at scale. These include:

- User management – registration, authentication, and account handling.
- Product management – product listings and catalog operations.
- Cart management – shopping cart creation and item operations.
- Order processing – order creation, tracking, and coordination with other services.
- Inventory management – tracking product availability.
- Payment processing – secure handling of payment operations.
- Notifications – order- and payment-related communication.
- Supporting concerns – APIs, databases, caching, asynchronous messaging, security, quality assurance, performance testing, and optimization.

The project was developed as a hypothetical/project-based system for internship learning and documentation. It does not represent a production e-commerce deployment.

6. Technologies and Tools Used

The following technologies and tools were used, proposed, or considered while designing and documenting the e-commerce platform. Not all tools were implemented in a running system; several were evaluated at the design level as part of the architectural exercise.

Technology / Tool	Purpose
Microservices Architecture	Architectural style used to divide the e-commerce platform into independent, loosely coupled services (proposed/considered for the design).
Spring Boot	Considered as the framework for implementing individual Java-based microservices due to its support for building REST APIs quickly.
REST APIs	Used as the communication mechanism between the frontend, the API Gateway, and individual microservices.
PostgreSQL	Proposed as the relational database for storing structured, persistent data such as users, products, and orders.
Redis	Proposed as an in-memory cache to reduce database load for frequently accessed data such as product listings.
Docker	Considered for containerizing each microservice so that it can be built, shipped, and run consistently across environments.
Kubernetes	Considered for orchestrating containerized services, including scaling, deployment, and recovery from failures.
Git	Used for version control and tracking changes to the project documentation and design artifacts.
API Gateway	Proposed as the single entry point for client requests, responsible for routing requests to the appropriate microservice.
Message Broker	Considered for enabling asynchronous communication between services, such as order and notification events.
React	Proposed as the frontend framework for building the client-facing interface of the e-commerce platform.

Technology / Tool	Purpose
k6 / JMeter	Considered as performance testing tools for simulating load and measuring system behaviour under traffic.

7. System Architecture

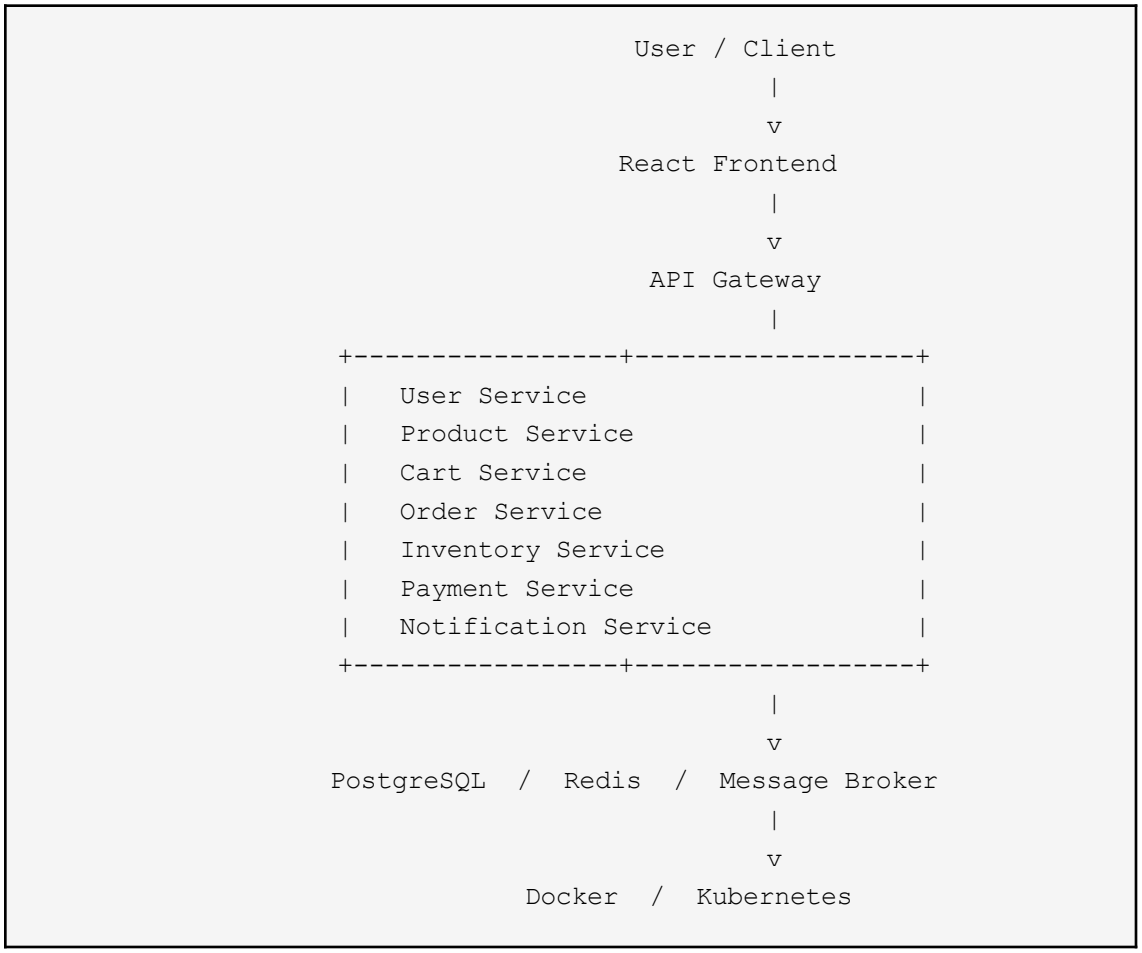
The proposed system follows a layered microservices architecture, where the React-based frontend communicates with the backend exclusively through an API Gateway. The API Gateway acts as the single entry point for all client requests, routing them to the appropriate microservice, and can also handle cross-cutting concerns such as authentication checks and rate limiting.

Each of the seven microservices — User, Product, Cart, Order, Inventory, Payment, and Notification — is designed to be independently deployable and responsible for a single business capability. This separation allows individual services to be scaled, updated, or restarted without affecting the rest of the platform. Services that need to coordinate, such as Order and Inventory, do so through well-defined APIs or asynchronous messages rather than sharing a database directly.

PostgreSQL was proposed for structured, persistent data due to its strong support for relational integrity, which is important for entities such as orders and inventory counts. Redis was proposed as a caching layer to reduce repeated database reads for frequently accessed data, such as product listings. A message broker was considered to support asynchronous communication — for example, allowing the Order Service to publish an event that the Notification Service consumes, without the two services being tightly coupled or waiting on each other synchronously.

Docker was considered for packaging each microservice into a consistent, portable container, while Kubernetes was considered for orchestrating these containers — handling deployment, horizontal scaling, and automatic recovery in case a service instance fails. From a security standpoint, the architecture assumes authentication and authorization at the API Gateway and service level, input validation at each service boundary, and secure handling of sensitive data such as payment information.

The proposed architecture is illustrated below:


User / Client
React Frontend
API Gateway
User Service
Product Service
Cart Service
Order Service
Inventory Service
Payment Service
Notification Service
PostgreSQL / Redis / Message Broker
Docker / Kubernetes

8. Methodology

The internship followed a structured four-stage methodology, with each stage building on the output of the previous one.

Stage 1: Architecture Design

A scalable microservices architecture was designed for the proposed e-commerce platform, including the identification of individual services, their responsibilities, supporting technologies, and security considerations.

Importance: it established the technical foundation on which all later planning and evaluation work depended.

Output / Analysis: an architecture diagram, service descriptions, and a list of supporting technologies and security measures.

Link to next stage: the architecture defined what needed to be built, which was used as the basis for the implementation plan in Stage 2.

Stage 2: Project Planning

The architecture from Stage 1 was converted into a practical implementation plan, including a task breakdown, resource considerations, a project timeline with milestones, and a risk management plan.

Importance: it demonstrated how a design is translated into an executable project rather than remaining only a conceptual diagram.

Output / Analysis: a task breakdown across services and phases, a timeline with milestones, and a list of key risks with mitigation approaches.

Link to next stage: the plan assumed that services would eventually be implemented, which made code quality — examined in Stage 3 — the next logical concern.

Stage 3: Code Review & Quality Assurance

A representative code review was carried out against typical implementations of the proposed microservices, identifying common issues such as weak authorization, insufficient input validation, and unsafe handling of secrets, along with recommended fixes.

Importance: it showed how systematic review can catch problems that would otherwise affect security, reliability, and maintainability after deployment.

Output / Analysis: a list of identified issues with impact ratings and recommendations, along with suggested QA test areas and a review checklist.

Link to next stage: a system that has been reviewed for correctness and quality is then reasonable to evaluate for performance, which is the focus of Stage 4.

Stage 4: Performance Testing & Optimization

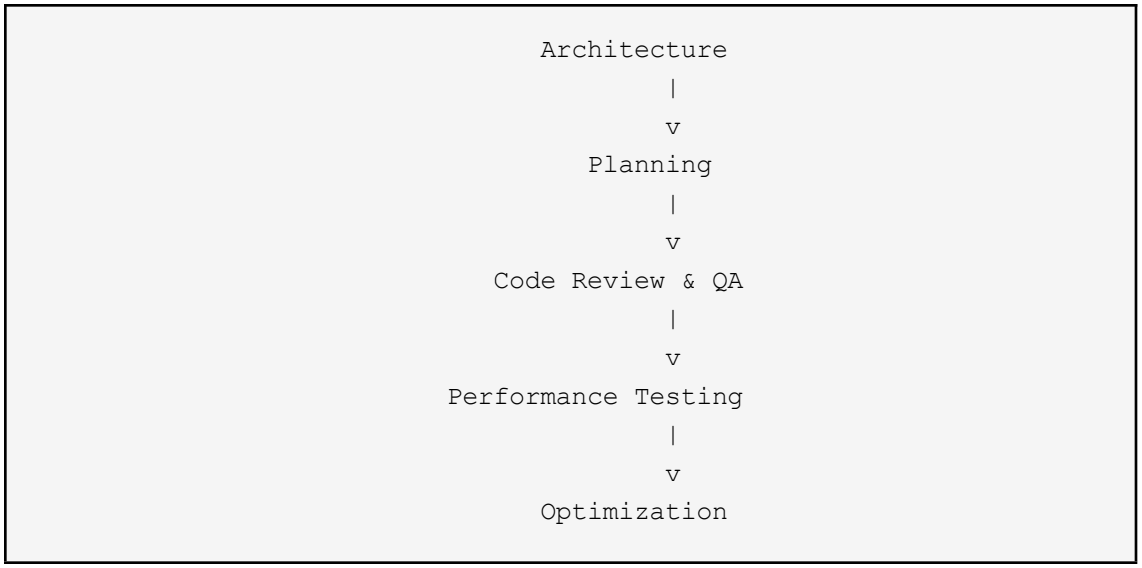
A performance testing and optimization strategy was designed, covering baseline, load, stress, spike, endurance, and scalability testing, along with key metrics and possible optimization methods.

Importance: it ensured that the architecture and implementation plan were evaluated against realistic high-traffic conditions rather than assumed to be adequate.

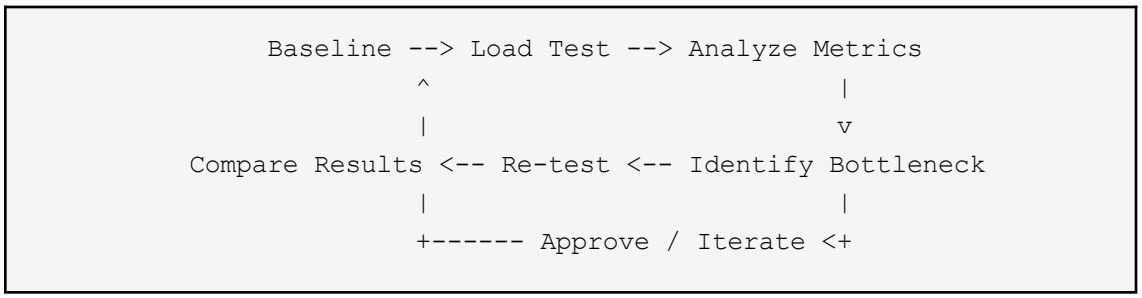
Output / Analysis: a set of test types, scenarios, metrics to monitor, likely bottlenecks, and optimization strategies aligned with a repeatable testing cycle.

Link to next stage: this stage completes the lifecycle by feeding performance findings back into architectural and implementation decisions.

The overall methodology flow is summarized below:



The performance optimization cycle used in Stage 4 followed a repeatable pattern of testing, analysis, and refinement:



9. Expected Outcomes

The internship is expected to result in the following learning and project outcomes. These represent expected/learning outcomes rather than measured production results.

- A better understanding of scalable software architecture and microservices design.
- Improved ability to plan a software project, including task breakdown and risk management.
- Awareness of secure coding practices relevant to authentication, authorization, and payment handling.
- A working understanding of code review and quality assurance processes.
- Familiarity with performance testing types and the metrics used to evaluate system behaviour.
- Improved ability to identify likely performance bottlenecks in a distributed system.
- Exposure to common optimization strategies such as caching, indexing, and horizontal scaling.
- A clearer understanding of scalability and reliability considerations in distributed systems.
- An overall improved understanding of the software engineering lifecycle, from design through to performance evaluation.

Bibliography / References

1. YuvaIntern. *About YuvaIntern – Job Simulation Internship Platform.* https://yuvaintern.com/about-us 僘
2. Spring. *Spring Boot Documentation.* https://docs.spring.io/spring-boot/ 僘
3. PostgreSQL Global Development Group. *PostgreSQL Documentation.* https://www.postgresql.org/docs/ 僘
4. Redis. *Redis Documentation.* https://redis.io/docs/latest/ 僘
5. Docker. *Docker Documentation.* https://docs.docker.com/reference/ 僘
6. Kubernetes. *Kubernetes Documentation.* https://kubernetes.io/docs/ 僘
7. Meta Open Source. *React Documentation.* https://react.dev/ 僘
8. Grafana Labs. *k6 Documentation.* https://grafana.com/docs/k6/ 僘